

หลักการทำ Feature Engineering และการคัดเลือก Feature

เปรียบเทียบระหว่าง clean_dataset.csv และ df_engineered

1. ภาพรวมและวัตถุประสงค์ (Overview)

กระบวนการ **Feature Engineering** เป็นการเตรียมและคัดเลือกข้อมูลเพื่อนำไปสอน Machine Learning Model โดยเป้าหมายหลักคือการทำนายว่าคำสั่งซื้อจะถูกส่งคืนหรือไม่ (is_returned) เราเริ่มต้นจากข้อมูลดิบที่ผ่านการทำความสะอาดแล้ว (clean_dataset.csv) นำมาแปลงรูปแบบ สร้างตัวแปรใหม่ และตัดตัวแปรที่ไม่จำเป็นออก จนได้ชุดข้อมูลสุดท้ายคือ df_engineered

2. กลุ่ม Feature ที่ถูกตัดทิ้ง (Dropped Features)

หลักการในการพิจารณาตัด Feature ทิ้ง แบ่งออกเป็น 4 ประเภทหลัก ได้แก่:

2.1. ข้อมูลระบุตัวตน (Identifiers & PII)

ตัวอย่าง: order_id, customer_id, customer_name, customer_phone, product_id, supplier_id

เหตุผล: ข้อมูลเหล่านี้เป็นข้อมูลเฉพาะเจาะจงของบุคคลหรือสิ่งของ ไม่ช่วยให้โมเดลเรียนรู้รูปแบบ (Pattern) ในภาพรวมได้ และหากนำไปสอนโมเดลอาจทำให้เกิดปัญหา Overfitting (จำข้อมูลได้แต่ทำนายของใหม่ไม่เป็น)

2.2. ข้อมูลในอนาคต หรือ ข้อมูลที่หลุดรอด (Data Leakage)

ตัวอย่าง: return_id, return_date, return_reason, item_condition, return_status, refund_amount, delivery_days

เหตุผล: ข้อมูลเหล่านี้จะเกิดขึ้นหรือรับรู้ได้ก็ต่อเมื่อ "มีการจัดส่งหรือคืนสินค้าไปแล้ว" การนำข้อมูลที่เกิดขึ้นทีหลังมาใช้ทำนายล่วงหน้าถือเป็นการโกง (Leakage) เพราะในโลกความเป็นจริงตอนที่ลูกค้าเพิ่งกดสั่งซื้อ เราจะยังไม่มีข้อมูลเหล่านี้เลย

2.3. ผลลัพธ์จากโมเดลเดิม (Model Outputs)

ตัวอย่าง: risk_score, risk_tier, shap_values

เหตุผล: เป็นผลลัพธ์ที่ได้จากการคำนวณของโมเดลตัวอื่น ไม่ใช่ข้อมูลดิบตั้งต้น หากนำมาใช้จะทำให้เกิดความซ้ำซ้อนและไม่ได้ประโยชน์เชิงพฤติกรรม

2.4. ข้อมูลวันที่ดิบๆ (Raw Dates)

ตัวอย่าง: order_date, expected_delivery_date

เหตุผล: โมเดล Machine Learning แบบตารางมักไม่สามารถเข้าใจวันที่ตรงๆ ได้ จึงถูกตัดออกและนำไปสกัดเป็นช่วงเวลา หรือระยะห่างแทน (เช่น แปลงเป็น order_hour, days_since_last_order)

3. กลุ่ม Feature ที่ถูกเก็บไว้และแปลงรูป (Kept & Transformed Features)

3.1. ข้อมูลการสั่งซื้อหลัก

ได้แก่: quantity, unit_price, total_amount, payment_method, channel_type, category

การแปลงรูป: มีการทำ Log Transformation (เช่น log_unit_price, log_total_amount) เพื่อลดผลกระทบจากข้อมูลที่มีราคาสูงผิดปกติ (Outliers) ทำให้กราฟการกระจายตัวของข้อมูลมีความสมดุลมากขึ้น

3.2. ข้อมูลส่วนลด (Discounts)

ได้แก่: promo_discount_pct

การแปลงรูป: สร้าง Flag แฉงเตือนเช่น is_high_discount (ส่วนลดมากกว่า 20%) เนื่องจากในเชิงธุรกิจพบว่าส่วนลดที่สูงมากมักกระตุ้นให้เกิดการซื้อแบบไม่ยั้งคิด และมีโอกาสขอคืนสินค้าในภายหลัง

4. กลุ่ม Feature ที่สร้างขึ้นใหม่ (Engineered Features) พร้อมที่มา

เรามีการสร้างข้อมูลใหม่จากความรู้เชิงธุรกิจ (Business Domain) โดยประกอบขึ้นมาจากข้อมูลเดิมที่มีอยู่ ดังนี้:

4.1. ข้อมูลสถิติย้อนหลัง (Point-in-Time Historical)

- total_orders_before (จำนวนออเดอร์ในอดีต)
- total_returns_before (จำนวนครั้งที่เคยคืน)
- customer_return_ratio (สัดส่วนการคืนของลูกค้า)
- return_rate_by_category (อัตราการคืนของหมวดหมู่นั้นๆ ในช่วง 90 วัน)

สร้างมาจาก: customer_id (เพื่อหาลูกค้าคนเดิม), category (หมวดหมู่สินค้า), order_date (ดูช่วงเวลา) และประวัติ is_returned ในอดีต

เหตุผล: คำนวณจากประวัติลูกค้าหรือสินค้า "ณ วันก่อนที่จะเกิดคำสั่งซื้อปัจจุบัน" เพื่อตรวจจับพฤติกรรมลูกค้าที่ชอบคืนสินค้าเป็นประจำ (Serial Returners) โดยรับประกันว่าจะไม่เกิด Data Leakage

4.2. ข้อมูลเชิงเวลา (Time-based)

- is_peak_hour (เวลาเร่งด่วนของการส่งของทางนั้นๆ)
- order_dayofweek (วันในสัปดาห์ 0-6)
- is_weekend (เป็นวันเสาร์-อาทิตย์หรือไม่)

สร้างมาจาก: สกัดค่าชั่วโมงและวันออกมาจาก order_date ส่วน is_peak_hour ใช้เงื่อนไขช่วงเวลา + channel_type

เหตุผล: ช่วยดักจับพฤติกรรมการณ์ซื้อ เช่น การซื้อดี ๆ อาจมีความเสี่ยงในการคืนมากกว่าซื้อตอนกลางวัน

4.3. เงื่อนไขทางธุรกิจและความเสี่ยง (Business Rules / Risk Flags)

- is_long_distance_cod (เก็บเงินปลายทางต่างจังหวัดไกลๆ)
- is_impulse_buy (ซื้อสินค้าแฟชั่นผ่านทีวี/TikTok ตอนกลางคืนดึกๆ)
- is_low_commitment (จ่ายปลายทางและมีส่วนลดสูงมาก)
- is_bracketing (ซื้อแฟชั่นที่ละหลายไซส์/หลายสีเพื่อลองแล้วคืน)

สร้างมาจาก: เป็นการผสมเงื่อนไข (IF-ELSE) ของข้อมูลเดิมหลายๆ ตัว เช่น

- province + payment_method == 'COD'
- category == 'Fashion' + channel_type + is_peak_hour
- payment_method == 'COD' + is_high_discount
- category == 'Fashion' + quantity > 1

เหตุผล: เป็นการจับคู่เงื่อนไขที่สะท้อนบริบททางธุรกิจ ซึ่งให้ประสิทธิภาพการทำนายที่แม่นยำกว่าการใช้ตัวแปรเดี่ยวๆ

4.4. การจับคู่ตัวแปร (Interaction Features)

- category_payment (หมวดหมู่ + วิธีการจ่ายเงิน)
- category_channel (หมวดหมู่ + ช่องทางการขาย)
- gender_province (เพศ + จังหวัด)

สร้างมาจาก: การนำข้อมูล Text 2 คอลัมน์มาต่อกันตรงๆ (String Concatenation)

เหตุผล: พฤติกรรมบางอย่างเกิดจากองค์ประกอบร่วมกัน เช่น การซื้อสินค้าแฟชั่นผ่านช่องทางทีวี (is_fashion_tv) อาจมีอัตราการคืนที่แตกต่างจากการซื้อเครื่องใช้ไฟฟ้าผ่านช่องทางเดียวกัน เป็นต้น

ข้อสรุปเพิ่มเติมประเด็นการใช้งาน History Data

คำถาม: บางข้อมูลเราแค่รู้ประวัติมันแต่ไม่ได้เอามาท่านายไข่ม้อย?

คำตอบ: ใช่ครับ สำหรับข้อมูล **ประวัติในอดีต (Historical Data)** เราไม่ได้เอาข้อมูลระดับ "บรรทัดต่อบรรทัด" ของอดีตมาโยนใส่โมเดลตรงๆ แต่เรานำประวัติเหล่านั้นมา **"สรุปเป็นสถิติ ณ จุดเวลาที่ลูกค้ากำลังจะซื้อ" (Point-in-Time)** เช่น แปลงประวัติในอดีตให้ออกมาเป็น "จำนวนที่เคยคืน" (``total_returns_before``)

ส่วนข้อมูลหลังบ้านของ **"คำสั่งซื้อปัจจุบัน" (Future/After-the-fact Data)** เช่น เหตุผลการคืน, วันที่ส่งของเสร็จ ของบิลรอบนี้ เราจะเก็บไว้ดูเป็น Report เฉยๆ แต่ห้ามเอามาใช้ทำนายบิลรอบนี้เด็ดขาด เพราะตอนที่ลูกค้าเพิ่งกดสั่งซื้อ เรายังไม่รู้ข้อมูลพวกนี้ครับ (หากใส่ไปจะทำให้เกิด Data Leakage)